Spring Security with Database Authentication (Spring Boot)

Detailed Implementation Notes

1. Objective

To move away from hardcoded credentials (in `application.properties` or Java Config) and implement a robust authentication system where user credentials are stored in a PostgreSQL database and verified using Spring Security.

2. Architecture Overview

- Controller: Handles HTTP requests.
- Service: Contains business logic (Custom `UserDetailsService`).
- Repository: Handles database interaction (Spring Data JPA).
- Model/Entity: Represents the database table (`Users`).
- Security Config: Configures `AuthenticationProvider` and `PasswordEncoder`.

---

3. Step-by-Step Implementation

Step 1: Add Dependencies (`pom.xml`)

You need Spring Data JPA for database connectivity and the PostgreSQL driver.

xml

```xml
<dependencies>
    <!-- Existing Spring Boot Web & Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- NEW: JPA & Database -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

Step 2: Database Configuration (`application.properties`)

Configure the connection to your PostgreSQL database.

properties

```
# Database Connection
spring.datasource.url=jdbc:postgresql://localhost:5432/tesco_one
spring.datasource.username=postgres
spring.datasource.password=0

# JPA Settings (Optional but recommended)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Step 3: Database Setup (SQL)

Create the database and table manually (or let Hibernate do it, but manual is shown in the video).

Sql

```sql
- Create Database
CREATE DATABASE tesco_one;

-- Create Table
CREATE TABLE users (
   id INT PRIMARY KEY,
   username TEXT,
   password TEXT
);

-- Insert Data (Passwords are currently Plain Text for this tutorial)
INSERT INTO users VALUES (1, 'navin', 'n@123');
INSERT INTO users VALUES (2, 'sachin', 's@123');
```

Step 4: Create Model Entity (`model/Users.java`)

This class maps to the `users` table in the database.

```java
package com.example.demo.model;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class Users {
   @Id
   private int id;
   private String username;
   private String password;

   // Getters and Setters
   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   public String getUsername() { return username; }
   public void setUsername(String username) { this.username = username; }
   public String getPassword() { return password; }
   public void setPassword(String password) { this.password = password; }
}
```

Step 5: Create Repository (`repo/UserRepo.java`)

Interface to interact with the database. Spring Data JPA creates the implementation automatically.

```java
package com.example.demo.repo;

import com.example.demo.model.Users;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepo extends JpaRepository<Users, Integer> {
   // Custom query method to find user by username
   Users findByUsername(String username);
}
```

Step 6: Create User Principal (`model/UserPrincipal.java`)

Spring Security requires a `UserDetails` object. We create a wrapper class to convert our `Users` entity into a format Security understands.

java

```java
package com.example.demo.model;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import java.util.Collection;
import java.util.Collections;

public class UserPrincipal implements UserDetails {
   private Users user;

   public UserPrincipal(Users user) {
       this.user = user;
   }

   @Override
   public Collection<? extends GrantedAuthority> getAuthorities() {
       // Assign default role 'USER'
       return Collections.singleton(new SimpleGrantedAuthority("USER"));
   }

   @Override
   public String getPassword() {
       return user.getPassword();
   }

   @Override
   public String getUsername() {
       return user.getUsername();
   }

   // Account status flags (Simplified for tutorial)
   @Override public boolean isAccountNonExpired() { return true; }
   @Override public boolean isAccountNonLocked() { return true; }
   @Override public boolean isCredentialsNonExpired() { return true; }
   @Override public boolean isEnabled() { return true; }
}
```

Step 7: Create Custom UserDetailsService (`service/MyUserDetailsService.java`)

This service loads user data from the DB during login.

java

```java
package com.example.demo.service;

import com.example.demo.model.UserPrincipal;
import com.example.demo.model.Users;
import com.example.demo.repo.UserRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class MyUserDetailsService implements UserDetailsService {

   @Autowired
   private UserRepo repo;

   @Override
   public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
       Users user = repo.findByUsername(username);

       if (user == null) {
           System.out.println("User Not Found");
           throw new UsernameNotFoundException("User not found");
       }

       return new UserPrincipal(user);
   }
}
```

Step 8: Configure Security (`config/SecurityConfig.java`)

Tell Spring Security to use the Database Provider (`DaoAuthenticationProvider`) instead of the default in-memory one.

Java

```java
package com.example.demo.config;

import com.example.demo.service.MyUserDetailsService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationProvider;
import
org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```java
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecuri
ty;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.NoOpPasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private MyUserDetailsService userDetailsService;

    @Bean
    public AuthenticationProvider authProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        // WARNING: NoOp is for learning only. Use BCrypt in production.
        provider.setPasswordEncoder(NoOpPasswordEncoder.getInstance());
        return provider;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        return http.csrf(customizer -> customizer.disable())
                .authorizeHttpRequests(request -> request
                        .requestMatchers("login", "register").permitAll()
                        .anyRequest().authenticated())
                .formLogin().and()
                .build();
    }
}
```

4. Testing & Output

Tool: Postman

- Method: GET
- URL: `http://localhost:8080/students` (or any secured endpoint)
- Authorization: Basic Auth
- Username: `navin`
- Password: `n@123`

✅ Success Output

- Status: `200 OK`
- Response: The JSON data from the secured endpoint (e.g., list of students).

- Console Log: `Hibernate: select ... from users ...` (Shows DB query executed).

❌ **Failure Output (Wrong Password)**

- Status: `401 Unauthorized`
- Response: `Unauthorized` or Login Page redirected.
- Reason: Password in DB (`n@123`) does not match input.

5. Common Errors & Troubleshooting

Scenario	Error / Output	Reason	Solution
Wrong DB Credentials	`Application failed to start: Connection refused`	`application.properties` has wrong password/URL.	Verify PostgreSQL username/password and port (5432).
Missing `@Service`	`NoSuchBeanDefinitionException`	Spring cannot find the `MyUserDetailsService` bean.	Ensure `@Service` annotation is on the class.
Missing Repo Method	`Compilation Error` or `QueryException`	`findByUsername` not defined in `UserRepo`.	Add `Users findByUsername(String username);` to Interface.
Password Mismatch	`401 Unauthorized` (Correct User, Wrong Pass)	DB has plain text, but Encoder expects Hash (or vice versa).	Ensure `NoOpPasswordEncoder` matches plain text DB. Switch to BCrypt for production.
Table Not Found	`Table "users" does not exist`	Database table not created.	Run the SQL `CREATE TABLE` script or set `ddl-auto=create`.

Package Import Error	`Cannot resolve symbol 'Entity'`	Missing JPA dependencies or wrong imports.	Ensure `spring-boot-starter-data-jpa` is in `pom.xml`.

6. Important Security Warning (From Transcript)

The video uses `NoOpPasswordEncoder.getInstance()`.

- What it does: Stores/compares passwords in Plain Text.
- Risk: If the database is hacked, all passwords are visible.
- Production Fix: Use `BCryptPasswordEncoder`.

java

```
// In SecurityConfig
provider.setPasswordEncoder(new BCryptPasswordEncoder());
```

- *Note: If you switch to BCrypt, you must hash the passwords in the database before inserting them, or registration logic must hash them before saving.*

7. Summary of Flow

1. User sends Login Request (Username/Password).
2. UsernamePasswordAuthenticationFilter catches the request.
3. AuthenticationManager delegates to DaoAuthenticationProvider.
4. DaoAuthenticationProvider calls UserDetailsService (`loadUserByUsername`).
5. UserDetailsService queries Repository -> Database.
6. Database returns User Entity.
7. UserDetailsService wraps Entity in UserPrincipal (`UserDetails`).
8. AuthenticationProvider compares Passwords.
9. Success: Session created, 200 OK.
10. Fail: 401 Unauthorized.